

Chatterbox Lab (CodeGrade)

 <https://github.com/learn-co-curriculum/python-p4-chatterbox-lab>  <https://github.com/learn-co-curriculum/python-p4-chatterbox-lab/issues/new>

Learning Goals

- Create an API with Flask for a React frontend application.

Introduction

So far, we've seen how to build a Flask API and perform various CRUD actions using SQLAlchemy. In this lab, you'll work on creating an API in Flask once more — but this time, you'll also have code for a React frontend application, so you can get a taste of full-stack development!

This project is separated into two applications:

- A React frontend, in the `client` directory.
- A Flask backend, in the `server` directory.

All of the features for the React frontend are built out, and we have a simple `json-server` API that you can run to see what the completed version of the app will look like. Your main goal with this lab is to build out a Flask API server to replace `json-server`, so most of your coding will be done in the backend.

Frontend Setup

Let's take a quick tour of what we have so far.

To get started, `cd` into the `client` directory. Then run:

```
$ npm install  
$ npm run server
```

This will install the React project dependencies, and run a demo API server using `json-server` . Next, run this in a new terminal:

```
$ npm start
```

NOTE: If you get an error message about "Error: digital envelope routines::unsupported", type the following in your terminal: `export NODE_OPTIONS=--openssl-legacy-provider` , then try starting the client again.

Then visit <http://localhost:3000>  (<http://localhost:3000>) in the browser and interact with the demo application to get a sense of its features.

Here's a demo of the what the React app should look like when using `json-server` as the API:



Chatterbox



Search...

Liza 9:15:27 AM

Hello 🖐️

Duane 9:16:37 AM

Hi brother

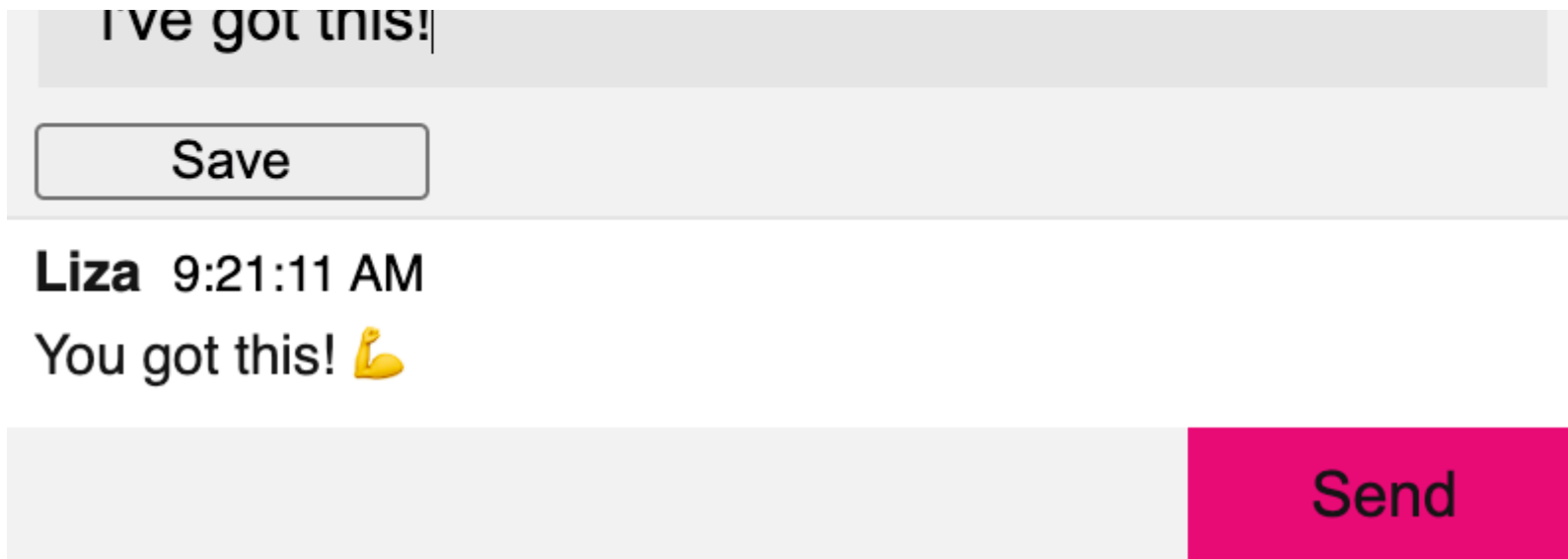
Liza 9:16:50 AM

let's get this chat app working



Duane 9:20:10 AM

let's get this



Take a look at the components provided in the `client` directory. Explore the code and pay special attention to where the React application is interacting with `json-server`. Where are the `fetch` requests being written? What routes are needed to handle these requests? What HTTP verbs? What data is being sent in the body of the requests?

Once you've familiarized yourself with the code, turn off `json-server` with `control + c` in the terminal where we ran `npm run server` (you can keep the React application running, though). Next, let's see what we have in the backend.

Backend Setup

In another terminal, run `pipenv install; pipenv shell` to install the dependencies and enter your virtual environment, then `cd` into the `server` directory to start running your Python code.

```
pipenv install && pipenv shell
cd server
```

In this directory, you're given a bare-bones template for a Flask API application. It should look familiar to other Flask labs you've seen and has all the code set up so you can focus on building out your model and API routes.

Note the database has not been created, nor have any migrations been performed.

You'll be responsible for:

- Implementing the `Message` model and performing migrations.
- Setting up the necessary routes to handle requests.
- Performing CRUD actions with SQLAlchemy.
- Sending the necessary JSON data in the responses.

Allowing Frontend Requests: CORS

The only new code for the server is the [Flask CORS extension](https://flask-cors.readthedocs.io/en/latest/) . This extension provides some Flask middleware which we need to configure so that applications running in the browser, like our React client, can make requests to the backend.

If we didn't use this gemextension, any requests from our React frontend in the browser would result in an error message like this:

```
Access to fetch at 'http://localhost:5000/messages' from origin
'http://localhost:3000' has been blocked by CORS policy: No
'Access-Control-Allow-Origin' header is present on the requested resource. If an
opaque response serves your needs, set the request's mode to 'no-cors' to fetch
the resource with CORS disabled.
```

The reason for this warning message is due to a browser security feature known as [Cross-Origin Resource Sharing \(CORS\)](https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS) . When we use JavaScript from one domain (aka origin) to make a request to a server on a different domain, the default behavior of the browser is to block those requests from going through.

For example, if I own the website `definitelynotahacker.com`, I can't use JavaScript to make a network request to `api.yourbankaccount.com`, unless `api.yourbankaccount.com` explicitly gives permission to my website.

To give that permission, any server that we want to make requests to using JavaScript must add some special **headers** to the response that tell the browser that the request was permitted.

Here's what the CORS configuration looks like (in the `server/app.py` file):

```
# server/app.py

from flask import Flask
from flask_cors import CORS

app = Flask(__name__)
CORS(app)
```

You don't have to make any changes to this configuration to complete this lab, but CORS warnings are a very common thing to encounter in web development, so next time you see them, you'll know what this means!

NOTE: There may come a time when you want CORS configured for some routes but not others. You can specify these with the optional `resources` argument or by instead using the `@cross_origin()` decorator on specific routes.

Different Types of Input

In previous lessons, we have used **form** data to retrieve input from the client. This is the typical approach we would take to this task, but some sites traffic in raw JSON instead. We're going to give that a shot here.

With the client running, navigate to Postman and point it to `localhost:3000`. Instead of using "Params", we will click on "Body", select "raw" from the radio buttons, then select "JSON" from the dropdown menu on the right.

Params Authorization Headers (7) Body Pre-request Script Tests Settings

[Cookies](#)

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON**

[Beautify](#)

1 |

From here, you can start to add messages:

```
{  
  "body": "Hello, World!",  
  "username": "Ian"  
}
```

When your Flask application is up and running, you can retrieve this data as a dictionary with the `request.get_json()` method.

Instructions

Work through the deliverables below. There are tests in the `server` folder. You'll need to `cd` into the `server` directory and run `pytest -x` to run the tests for the Flask backend until the first failure.

Make sure to try out your routes from the React frontend application as well once you have everything set up. You can run your Flask server from the `server/` directory with:

```
$ python app.py
```

Or, if you have configured your Flask environment:

```
$ flask run
```

Model

Start by generating the `Message` model and the necessary migration code to create messages with the following attributes:

- "body": String.
- "username": String.
- "created_at": DateTime.
- "updated_at": DateTime.
- Don't forget to add default values for "created_at" and "updated_at"!
 - (Hint - we discussed this in the Phase 3 Many-to-Many Relationships reading and gave an example in the Phase 4 Building a Get API Reading.)

Once you've created the model, you should initialize the database, generate and run the migrations, and use the provided `seed.py` file to seed the database:

```
$ flask db init
$ flask db revision --autogenerate -m'your message'
$ flask db upgrade
$ python seed.py
```

Routes

Build out the following routes to handle the necessary CRUD actions:

- `GET /messages` : returns an array of all messages as JSON, ordered by `created_at` in ascending order.
- `POST /messages` : creates a new message with a `body` and `username` from params, and returns the newly created post as JSON.

- **PATCH** `/messages/<int:id>` : updates the **body** of the message using params, and returns the updated message as JSON.
 - **DELETE** `/messages/<int:id>` : deletes the message from the database.
-

Resources

- [Flask - Pallets](https://flask.palletsprojects.com/en/2.2.x/)  `(https://flask.palletsprojects.com/en/2.2.x/)`
- [Cross-Origin Resource Sharing - Mozilla](https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS)  `(https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS)`
- [Flask-CORS](https://flask-cors.readthedocs.io/en/latest/)  `(https://flask-cors.readthedocs.io/en/latest/)`
- [flask.json jsonify Example Code - Full Stack Python](https://www.fullstackpython.com/flask-json-serialize-examples.html)  `(https://www.fullstackpython.com/flask-json-serialize-examples.html)`
- [SQLAlchemy-serializer - PyPI](https://pypi.org/project/SQLAlchemy-serializer/)  `(https://pypi.org/project/SQLAlchemy-serializer/)`

This tool needs to be loaded in a new browser window

Load Chatterbox Lab (CodeGrade) in a new window